Internship Project Report

Project: Automated Power Management System
Platform: CODESYS V3.5 / Eaton XC-204

Powering Business Worldwide

Final Project Report
January 13, 2026

Intern: Joe Sfeir **Supervisor:** Mr. Fadel Essaily

1. Executive Summary

This project focused on the design, development, and simulation of an Automatic Transfer Switch (ATS) and Load Management System using the Eaton XC-204 PLC. The system autonomously manages a Utility feed and two backup Generators (100kVA and 200kVA) based on real-time power consumption data via Modbus RTU. The project successfully demonstrated logic for seamless power transfer, load-dependent generator selection, and a web-based Human-Machine Interface (HMI) for remote control and monitoring.

2. Project Scope & Objectives

The primary objective was to migrate from basic relay logic (EasySoft) to a structured IEC 61131-3 environment (CODESYS V3.5).

Preliminary Phase: The easySoft Foundation

Prior to commencing the CODESYS development, I executed two full-cycle automation projects using the **Eaton easyE4 Smart Relay** to get familiar with the fundamentals of industrial logic.

- **Project A: Duty/Standby/Assist Configuration System:** Designed a Duty/Standby/Assist configuration controlled by an analog pressure sensor.
  - **Analog Signal Processing:** Implemented Linear Scaling blocks to map raw 0-10V sensor signals to human-readable pressure values (0-16 Bar) on the display.
  - **Wear Leveling (Cyclic Alternation):** Engineered an algorithm that forces the Duty and Standby pumps to swap roles after every cycle to ensure even mechanical aging.
  - **HMI Configuration:** Created a menu system allowing facility managers to adjust "Cut-In" and "Cut-Off" setpoints directly via the easyE4 screen, eliminating the need for a laptop connection.
- **Project B: Pressure Control:** Developed logic for a 2-Duty / 1-Standby pump system with simultaneous operation requirements.
  - **State Management:** Unlike the simple threshold logic of the booster, this required implementing a Counter Mechanism to define specific system states (State 1: P1+P2, State 2: P2+P3).
  - **Redundancy:** Validated "Instant Failover" logic where the Standby pump engages immediately if a Duty pump trips, ensuring zero loss of cooling capacity.

3. System Architecture

3.1 Hardware Configuration

- **Controller:** Eaton XC-204 Modular PLC (CODESYS V3.5).
- **I/O System:** Eaton XN300 Slice I/O.
  - **XN-322-16DI-PD:** Monitoring Breaker Status and Generator Feedback.
  - **XN-322-16DO-P05:** Controlling Breaker Coils and Generator Start Signals.
- **Sensor:** Eaton APM Series Power Meter (Modbus RTU).

3.2 Software Design

The software was structured using Object-Oriented principles:

- **Global Variable Lists (GVL):** Separated into `GVL_IO` (Hardware) and `GVL_HMI` (Visu).
- **Logic:** Implemented in Structured Text (ST) for complex decision-making and Continuous Function Chart (CFC) for signal flow.

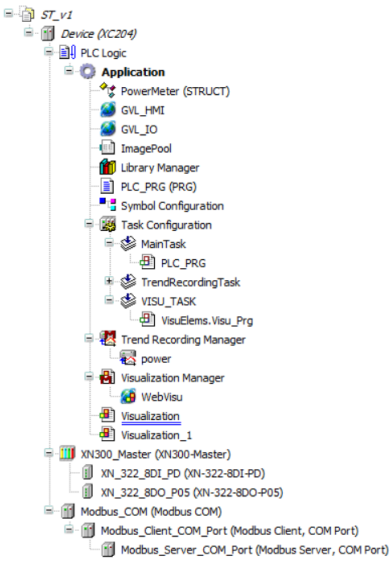

Figure 3.1: System Overview

4. Methodology & Implementation

4.1 Communication

I configured a Modbus Master on the PLC's COM port to poll the Power Meter.

- **Protocol:** Modbus RTU, 9600 Baud, 8N1.
- **Register Mapping:** Using the address table found on the meter's manual I mapped the parameters of interest directly to PLC variables.

4.2 Control Logic

The core logic resides in `PLC_PRG` and handles three critical states:

- **Sample & Hold (Memory):** Implemented a logic gate that "freezes" the load value the moment Utility fails. This prevents the PLC from reading "0 kW" during a blackout and making the wrong generator decision.
- **Hysteresis:** Defined separate "Switch Up" (90kW) and "Switch Down" (70kW) thresholds to prevent generator chattering during fluctuating loads.
- **Safety Interlocking:** Used a CASE statement architecture to ensure Mutual Exclusion, where it is physically impossible for the software to close the Utility and Generator breakers simultaneously.
- **Fault Detection:** Implemented a fault detection mechanism that is fed through a feedback signal from the generator. If for a specified time period after the start signal was given to the generator and the load was not being supplied, the Fault Alarm will turn on. Based on client requirements, the system will either wait for human intervention or switch to another source.

4.3 Visualization

Developed a WebVisu interface accessible via IP address:

- **Single Line Diagram (SLD):** Animated power lines that change color (Green/Red) based on the "Live" status of the busbar.
- **Dashboard:** Digital readout of Voltage, Current, and Power Factor.
- **Control Panel:** A "Manual/Auto" selector with a dynamic multi-state button for manual generator testing.

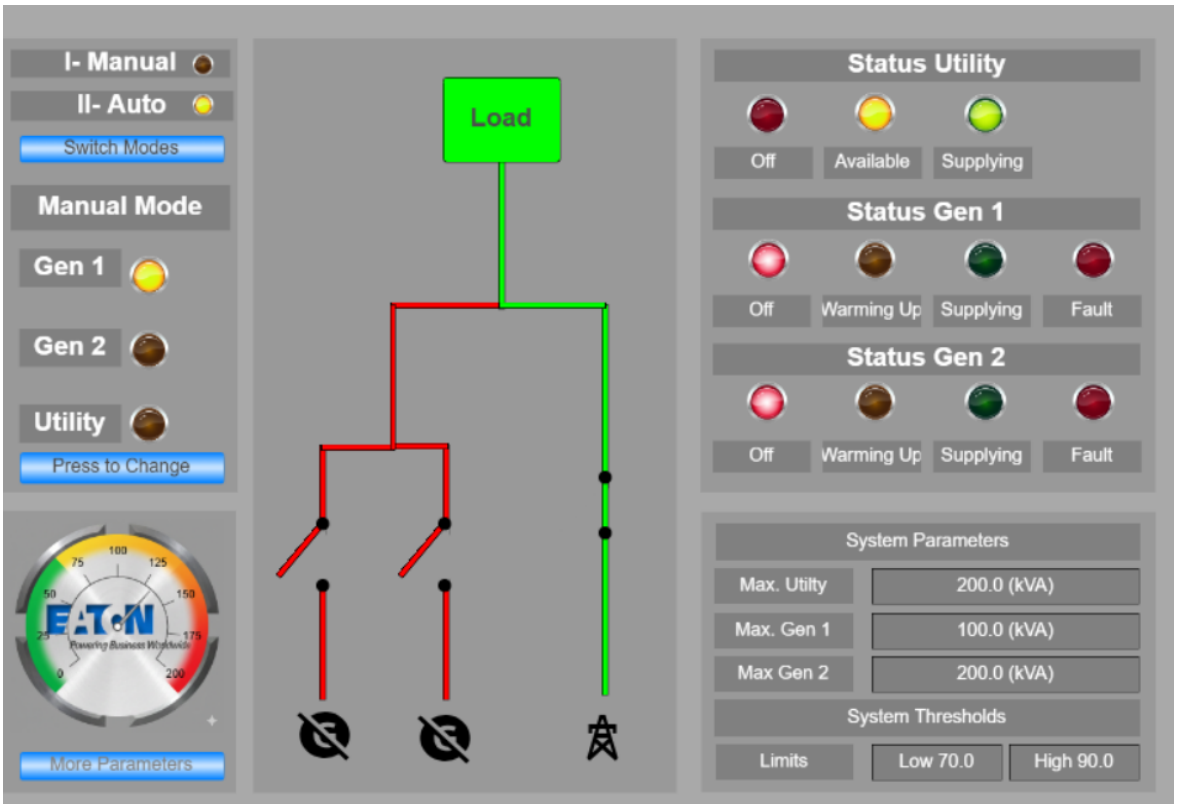

Figure 1: Web Visualization in Automatic Mode. The Single Line Diagram indicates the Utility is feeding the load (Green path).

Figure 2: Secondary Visualization Interface display additional detailed parameters.

5. Challenges & Solutions

- **Challenge 1: Optimization of Logic Flow**
  Issue: Implementing the sequential logic using graphical languages like FBD or CFC resulted in cluttered diagrams that were difficult to debug and maintain.
  Solution: I migrated the core decision-making logic to Structured Text (ST). Since the ATS algorithm relies heavily on conditional branching (`IF/ELSE`) and state selection (`CASE`), ST proved far more efficient. Using my prior experience with Python, I found the syntax natural to implement, significantly reducing code complexity and improving readability.
- **Challenge 2: Simulation vs. Reality**
  Issue: During simulation, the "Warmup" logic would fail because there was no physical generator to send a "Running" feedback signal.
  Solution: I added a "Simulation Helper" block. This code mimics the physical behavior of an engine by artificially delaying the feedback signal by 2 seconds, allowing full testing of the warmup timers and alarms without hardware.
- **Challenge 3: Manual Mode Conflict**
  Issue: Initially, the Automatic logic would override Manual commands if the Utility was present.
  Solution: Restructured the code to prioritize the "Operation Mode" check. If "Manual" is selected, the Automatic logic is bypassed entirely, giving the operator full control in addition to a discrete counter to change manual source selection.

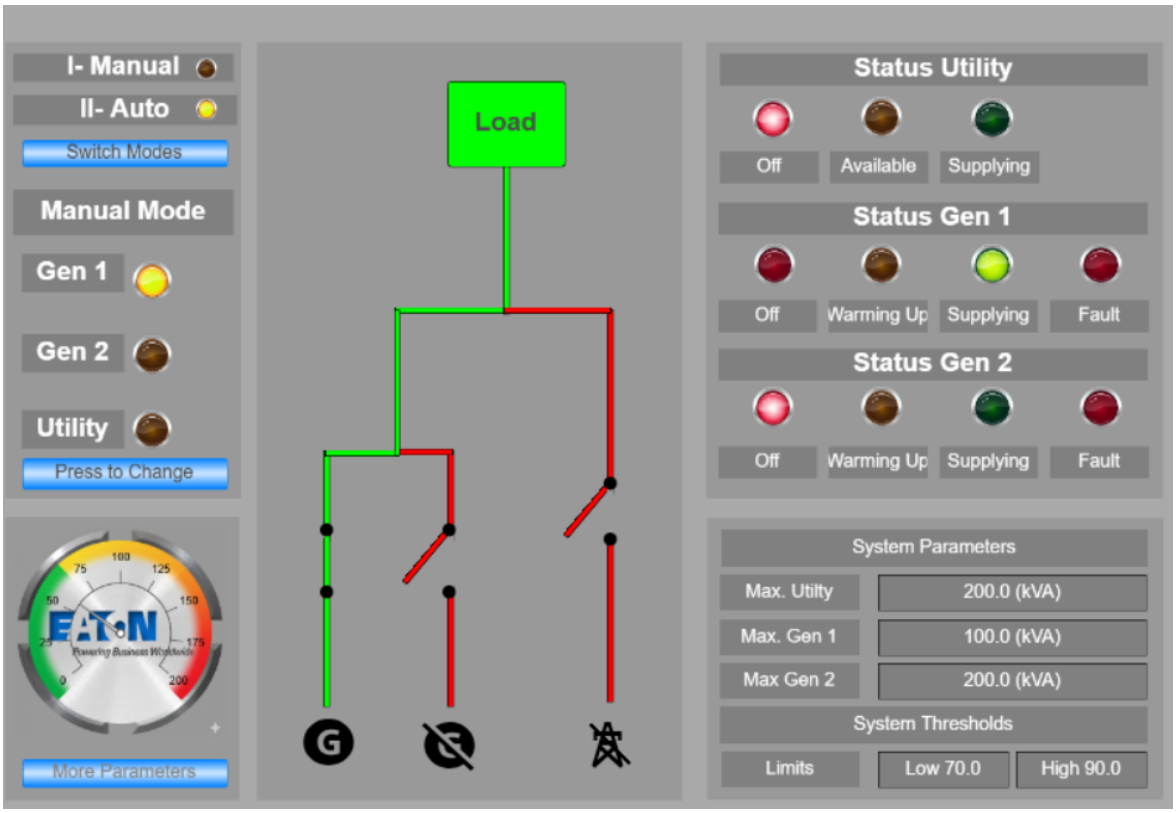

Figure 3: Simulation of a Utility Failure. The system has automatically started Generator 1 (based on the load), synchronized, and closed the breaker.

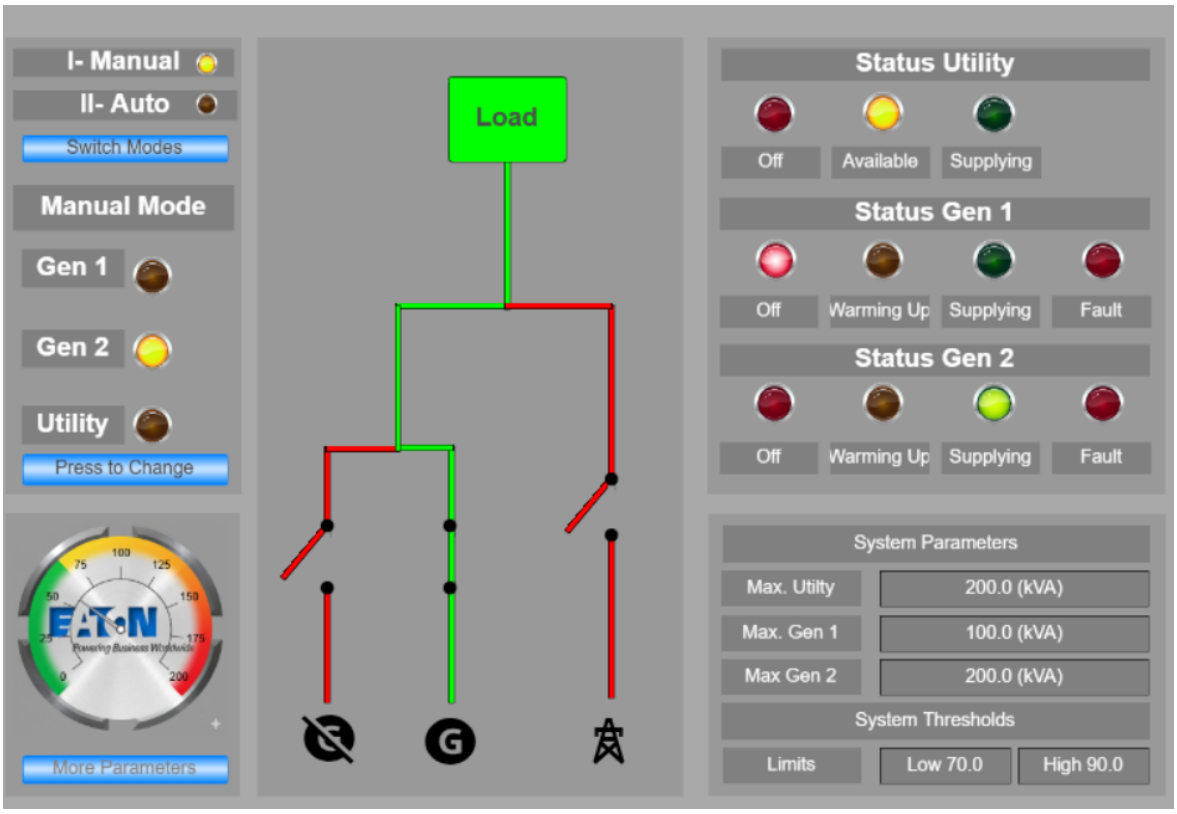

Figure 4: Simulation of Manual Mode. User switched over to manual mode with Gen 2 as the source. Even if Utility source (our priority) became available, it will not supply the source as long as Manual Mode is toggled.

6. Future Improvements

If I had more time to extend this project, I would implement:

- **Hybrid Solar & BMS Integration:** Since Solar PV capacity is often intermittent and insufficient to carry the full load, I would implement Source Co-Sharing. The logic would allow the PV Inverter to run in parallel with the Utility or Generator. Additionally, a Battery Management System (BMS) module would be integrated to monitor health and ensure batteries are charged/discharged optimally for night-time or blackouts.
- **AI-Driven Predictive Control:** I would integrate a function to perform pattern recognition on historical data. By analyzing the time-stamps of previous blackouts and warnings, the system could predict outages and pre-start the generator or pre-charge batteries.
- **Event Logging:** Currently, the system only shows active status. I would build a persistent History Logger. This would act as a scrollable "Black Box" recorder, color-coding messages by severity (Red for Alarms, Yellow for Warnings, Green for System State Changes) to help operators diagnose the root cause of failures. I would also implement a Notification System to email the messages to maintenance staff automatically.

7. Conclusion

This internship provided deep exposure to IEC 61131-3 programming. Moving from EasySoft to CODESYS required a shift from simple relay thinking to structured, algorithmic thinking. The resulting project is a robust, fail-safe power management system that not only automates the switching process but provides operators with clear, real-time visibility into the plant's status. The system is ready for hardware deployment.

Acknowledgement: I extensively utilized CODESYS manuals and technical training courses from ABB, Topcon, and Beijer Electronics.